G101 BASES DE DATOS II

S1. TAREA. EXPLORADORES NOSQL. VALOR 15%.

SANTIAGO PADIERNA SUAREZ

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
FACULTAD DE INGENIERÍA

ORALIA CORTES GRAJALES

TECNOLOGÍA EN DESARROLLO DE SOFTWARE

2026-2

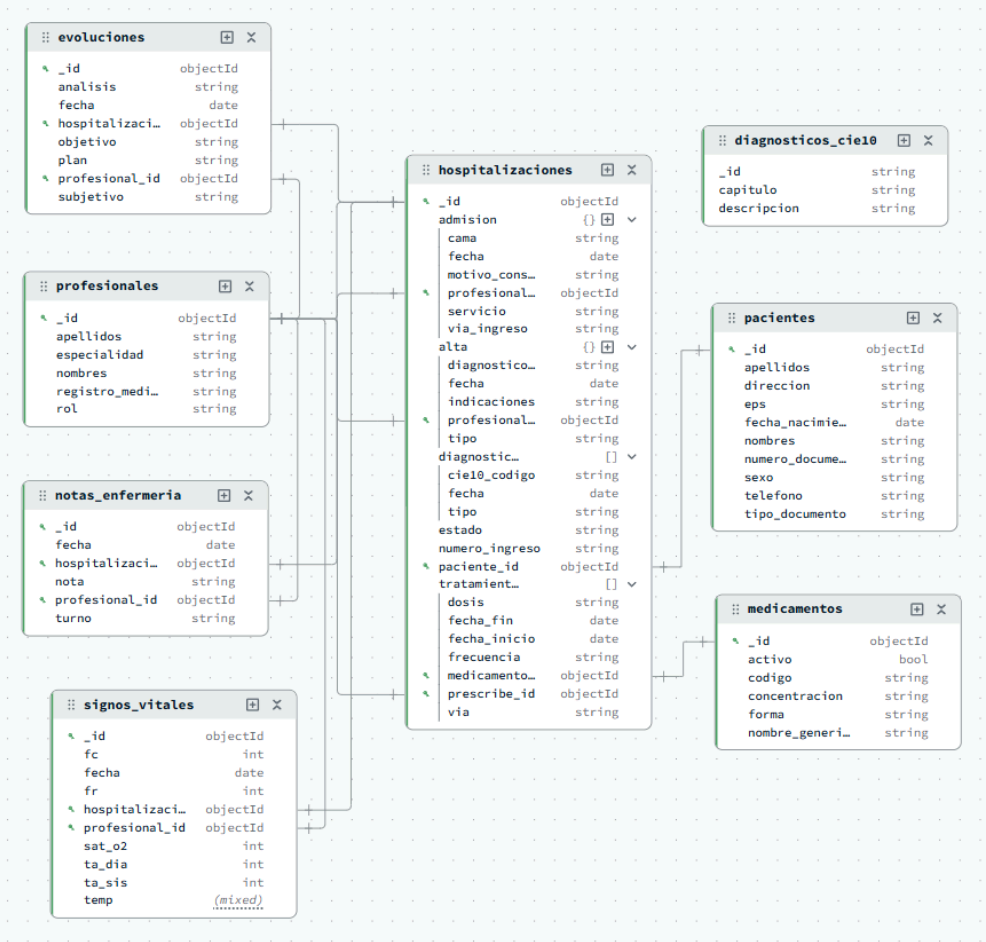
0. Anotaciones

Video: https://drive.google.com/drive/u/0/folders/1Gx7LYmhUczAqEj3rQFJa-Z8s3JD6_6hn

Github: https://github.com/soypadierna/hce-hospitalizacion-mongodb

1. Esquema del modelo

Modelo documental híbrido para el proceso de hospitalización de una Historia Clínica Electrónica (piloto Gobernación de Antioquia).



pacientes (datos identificables, aislados)

Campo	Tipo	Oblig.	Modo
tipo_documento, numero_documento	enum / string	Sí	Identificador único (índice único)
nombres, apellidos	string	Sí	Solo en esta colección
fecha_nacimiento	date	Sí	
sexo	enum (M, F, O)	Sí	
telefono, direccion, eps	string	No	Contacto

hospitalizaciones (datos clínicos)

Campo	Tipo	Oblig.	Modo
paciente_id	ObjectId	Sí	Referenciado
numero_ingreso	string	Sí	Índice único
estado	enum (activa, alta)	Sí	
admision	objeto	Sí	Embebido: tamaño acotado, se lee siempre junto
diagnosticos[]	array (1 a 20)	Sí	Embebido; cie10_codigo referenciado
tratamientos[]	array (máx. 100)	No	Embebido; medicamento_id y prescribe_id referenciados
alta	objeto	No	Embebido: único y acotado
eliminado	bool	No	Soft delete: solo existe si el registro fue eliminado lógicamente; las consultas filtran `eliminado: { $ne: true }`
fecha_eliminacion	date	No	Se agrega junto con `eliminado`

motivo_eliminacion	string	No	Se agrega junto con `eliminado`; deja trazabilidad

signos_vitales, notas_enfermeria, evoluciones (crecimiento sin tope, referenciadas)

Campo	Tipo	Oblig.	Modo
hospitalizacion_id	ObjectId	Sí	Referencia al padre
fecha	date	Sí	
profesional_id	ObjectId	Sí	Referencia al catálogo
fc, fr, temp, sat_o2, ta_sis, ta_dia	number	Sí (signos_vitales)	
nota, turno	string / enum	Sí (notas_enfermeria)	
subjetivo, objetivo, analisis, plan	string	Sí (evoluciones)	Formato SOAP

Catálogos referenciados

Colección	Campos	Clave
medicamentos	codigo, nombre_generico, forma, concentracion, activo	codigo único
profesionales	registro_medico, nombres, apellidos, rol, especialidad	registro_medico único
diagnosticos_cie10	descripcion, capitulo	_id = código CIE-10
camas	servicio, estado (enum: libre, ocupada, mantenimiento)	_id = código de cama (ej. M-12)

2. Creación de BD y Carga de datos

Script:
https://drive.google.com/file/d/1qJlq7ZAyGm8YEahm5AzE50lyX9nBz3SV/view?usp=drive_link

```
// hce_seed.js
    ["2026-10-09T09:00", pr[4], "manana", "Refiere dolor lumbar leve, se administra analgésico
según orden."],
    ["2026-10-10T22:00", pr[6], "noche", "Duerme sin fiebre, diuresis espontánea clara."]]);
  // Hospitalización 5: EPOC exacerbado (Pedro Pablo), con alta
  hs(ho[4], pa[4], "HOS-2026-0005", "alta",
    ad("2026-10-12T05:50", "urgencias", "Neumología", "N-03", "Disnea y sibilancias de 2 días",
pr[3]),
    dx([["J44.1", "ingreso", "2026-10-12T06:20"], ["I10", "secundario", "2026-10-12T09:00"], ["J44.1",
"principal", "2026-10-12T09:00"], ["J44.1", "egreso", "2026-10-17T10:00"]]),
    tx([[me[7], "2 inhalaciones", "inhalada", "cada 4 h", "2026-10-12T06:30", "2026-10-17T06:30",
pr[2]], [me[10], "8 mg", "IV", "cada 24 h", "2026-10-12T07:00", "2026-10-16T07:00", pr[2]], [me[1],
"500 mg", "oral", "cada 24 h", "2026-10-12T09:30", "2026-10-15T09:30", pr[2]]]),
    al("2026-10-17T11:00", "mejoria", "J44.1", "Inhaladores de mantenimiento, abstinencia de tabaco
y control en 10 días", pr[2]));
  ev(ho[4], [
    ["2026-10-12T09:00", pr[2], "Disnea marcada con sibilancias y tos con esputo.", "FR 28, SatO2
88% en aire ambiente, sibilancias difusas.", "EPOC exacerbado.", "Broncodilatador, corticoide,
oxígeno controlado y antibiótico."],
    ["2026-10-14T08:00", pr[2], "Disnea menor, tos productiva.", "SatO2 93% con oxígeno a 2 L/min,
sibilancias escasas.", "Mejoría parcial.", "Continuar manejo y retirar oxígeno según tolerancia."],
    ["2026-10-16T08:00", pr[2], "Sin disnea en reposo.", "SatO2 94% en aire ambiente, sin
sibilancias.", "Exacerbación resuelta.", "Educación en inhaladores y egreso al día siguiente."]]);
  sv(ho[4], [
    ["2026-10-12T07:00", pr[4], 110, 28, 37.2, 88, 148, 90], ["2026-10-12T15:00", pr[5], 104, 26, 37,
90, 144, 88],
    ["2026-10-13T08:00", pr[4], 96, 24, 36.9, 92, 140, 86], ["2026-10-14T08:00", pr[4], 90, 22, 36.8,
93, 136, 84],
    ["2026-10-15T08:00", pr[4], 84, 20, 36.7, 94, 132, 82], ["2026-10-16T08:00", pr[4], 80, 18, 36.6,
94, 128, 80]]);
  nt(ho[4], [
    ["2026-10-12T08:00", pr[4], "manana", "Se inicia oxígeno por cánula y nebulización, ansiedad
leve por la disnea."],
    ["2026-10-13T14:00", pr[5], "tarde", "Tolera dieta, expectoración mucosa moderada."],
    ["2026-10-15T21:00", pr[6], "noche", "Duerme sin oxígeno suplementario, sin disnea
nocturna."]]);
  // Hospitalización 6: cetoacidosis diabética (Valentina), con alta
  hs(ho[5], pa[5], "HOS-2026-0006", "alta",
    ad("2026-10-15T21:40", "urgencias", "Medicina interna", "M-09", "Poliuria, polidipsia y vómito de
24 horas", pr[3]),
    dx([["E11.1", "ingreso", "2026-10-15T22:10"], ["A09", "secundario", "2026-10-15T22:10"],
["E11.1", "principal", "2026-10-16T08:00"], ["E11.1", "egreso", "2026-10-20T10:00"]]),
    tx([[me[11], "0.1 UI/kg/h", "IV", "infusión continua", "2026-10-15T22:30", "2026-10-16T20:00",
pr[0]], [me[5], "20 UI", "SC", "cada 24 h", "2026-10-16T20:30", "2026-10-20T11:00", pr[0]], [me[6],
"20 mg", "oral", "cada 24 h", "2026-10-16T09:00", "2026-10-20T09:00", pr[0]]]),
    al("2026-10-20T11:00", "mejoria", "E11.1", "Control con endocrinología, insulina basal y
educación en glucometría", pr[0]));
  ev(ho[5], [
```

```
  ["2026-10-16T08:00", pr[0], "Náuseas, sed intensa, mejoría parcial del dolor abdominal.",
"Respiración profunda, mucosas secas, glucemia 410 mg/dL.", "Cetoacidosis diabética en
manejo.", "Infusión de insulina, líquidos IV y control de gases y electrolitos."],
  ["2026-10-17T08:00", pr[0], "Sin vómito, tolera líquidos.", "Glucemia 180 mg/dL, anión gap
normalizado.", "Cetoacidosis resuelta.", "Transición a insulina subcutánea."],
  ["2026-10-19T08:00", pr[0], "Asintomática, apetito conservado.", "Glucemias entre 110 y 160
mg/dL, signos vitales estables.", "Diabetes controlada con esquema basal.", "Educación
diabetológica y egreso programado."]]);
 sv(ho[5], [
  ["2026-10-15T22:30", pr[6], 118, 28, 37, 97, 98, 60], ["2026-10-16T02:00", pr[6], 110, 26, 36.9,
98, 102, 64],
  ["2026-10-16T08:00", pr[4], 100, 22, 36.8, 98, 108, 68], ["2026-10-17T08:00", pr[4], 88, 18, 36.7,
99, 112, 70],
  ["2026-10-18T08:00", pr[5], 82, 16, 36.6, 99, 114, 72], ["2026-10-19T08:00", pr[4], 78, 16, 36.6,
99, 116, 74]]);
 nt(ho[5], [
  ["2026-10-15T23:00", pr[6], "noche", "Se instala acceso venoso y líquidos IV, glucometrías
horarias."],
  ["2026-10-16T10:00", pr[4], "manana", "Persisten náuseas leves, se mantiene en ayuno y
monitoreo de glucemia."],
  ["2026-10-18T15:00", pr[5], "tarde", "Inicia dieta para diabético, tolera completa."]]);
 // Verificación de integridad referencial y de estado
 const hex = x => x.toHexString();
 const chk = { "Hospitalizaciones con paciente_id inexistente": [], "Tratamientos con
medicamento_id inexistente": [], "Diagnósticos con cie10_codigo ausente del catálogo": [],
"Registros hijos con hospitalizacion_id huérfano": [], "Estado inconsistente con objeto alta": [],
"Camas inexistentes o activas sin cama ocupada": [] };
 const kP = "Hospitalizaciones con paciente_id inexistente", kM = "Tratamientos con
medicamento_id inexistente", kD = "Diagnósticos con cie10_codigo ausente del catálogo", kH =
"Registros hijos con hospitalizacion_id huérfano", kE = "Estado inconsistente con objeto alta", kC =
"Camas inexistentes o activas sin cama ocupada";
 const idsPac = new Set(db.pacientes.distinct("_id").map(hex));
 const idsMed = new Set(db.medicamentos.distinct("_id").map(hex));
 const idsHos = new Set(db.hospitalizaciones.distinct("_id").map(hex));
 const codigos = new Set(db.diagnosticos_cie10.distinct("_id"));
 const camasEstado = {};
 db.camas.find().forEach(c => { camasEstado[c._id] = c.estado; });
 db.hospitalizaciones.find().forEach(h => {
  if (!idsPac.has(hex(h.paciente_id))) chk[kP].push(h.numero_ingreso);
  (h.tratamientos || []).forEach(t => { if (!idsMed.has(hex(t.medicamento_id)))
chk[kM].push(h.numero_ingreso + " -> " + hex(t.medicamento_id)); });
  (h.diagnosticos || []).forEach(d => { if (!codigos.has(d.cie10_codigo))
chk[kD].push(h.numero_ingreso + " -> " + d.cie10_codigo); });
  if (h.estado === "activa" && h.alta) chk[kE].push(h.numero_ingreso + ": activa con alta");
  if (h.estado === "alta" && !h.alta) chk[kE].push(h.numero_ingreso + ": alta sin objeto alta");
  if (!(h.admision.cama in camasEstado)) chk[kC].push(h.numero_ingreso + ": cama " +
h.admision.cama + " inexistente");
  else if (h.estado === "activa" && camasEstado[h.admision.cama] !== "ocupada")
chk[kC].push(h.numero_ingreso + ": cama " + h.admision.cama + " no figura ocupada");
```

```
  });
  ["signos_vitales", "notas_enfermeria", "evoluciones"].forEach(c =>
db.getCollection(c).find().forEach(x => { if (!idsHos.has(hex(x.hospitalizacion_id))) chk[kH].push(c +
" " + hex(x._id)); }));
  Object.keys(chk).forEach(k => print(k + ": " + (chk[k].length ? "\n  - " + chk[k].join("\n  - ") : "OK")));
  // Conteo de documentos por colección
  print("--- Conteo de documentos ---");
  ["pacientes", "profesionales", "medicamentos", "diagnosticos_cie10", "camas", "hospitalizaciones",
"signos_vitales", "notas_enfermeria", "evoluciones"].forEach(c => print(c + ": " +
db.getCollection(c).countDocuments()));
})();
```

La base `hce_hospitalizacion` se crea y se puebla con el script `scripts/hce_seed.js`, que se pega completo en mongosh. El script se puede ejecutar varias veces: elimina las colecciones existentes antes de recrearlas.

Creación. El script crea las 9 colecciones (pacientes, profesionales, medicamentos, diagnósticos CIE-10, camas, hospitalizaciones, signos vitales, notas de enfermería y evoluciones) con validadores $jsonSchema y sus índices (únicos, compuestos, multikey y de texto).

Carga. Inserta datos ficticios coherentes: 6 pacientes, 7 profesionales, 12 medicamentos, 10 códigos CIE-10, 11 camas y 6 hospitalizaciones. Dos pacientes comparten el diagnóstico principal J18.9 y una hospitalización permanece activa (HOS-2026-0003), cuya cama (C-05) figura como ocupada mientras las demás quedan libres. Cada hospitalización incluye admisión, diagnósticos, 2 o más tratamientos, 3 evoluciones SOAP, 6 registros de signos vitales y 3 notas de enfermería. Las hospitalizaciones con alta incluyen además el objeto alta.

Verificación de integridad. El script comprueba la existencia de paciente y medicamento referenciados, la presencia de cada código CIE-10 en el catálogo, la existencia de la cama asignada a cada hospitalización y que las hospitalizaciones activas tengan su cama ocupada, la ausencia de registros hijos huérfanos y la coherencia entre estado y el objeto alta. Cada comprobación imprime “OK” o la lista de problemas.

```
< Hospitalizaciones con paciente_id inexistente: OK
< Tratamientos con medicamento_id inexistente: OK
< Diagnósticos con cie10_codigo ausente del catálogo: OK
< Registros hijos con hospitalizacion_id huérfano: OK
< Estado inconsistente con objeto alta: OK
< Camas inexistentes o activas sin cama ocupada: OK
< --- Conteo de documentos ---
< pacientes: 6
< profesionales: 7
< medicamentos: 12
< diagnosticos_cie10: 10
< camas: 11
< hospitalizaciones: 6
< signos_vitales: 36
< notas_enfermeria: 18
< evoluciones: 18
```

3. Crud

Script: https://drive.google.com/file/d/1HE8xRDbBYTjVs-edvaOSWdG7ES2nQlMp/view?usp=drive_link

```
// hce_crud.js
// Operaciones CRUD sobre registros de prueba (HOS-2026-0007).
// Requisito: haber ejecutado antes hce_seed.js. Al terminar, la base queda igual que tras el seed.
(function () {
  db = db.getSiblingDB("hce_hospitalizacion");
  const F = s => ISODate(s + ":00Z");
  const T = t => print("\n=== CRUD | " + t + " ===");
  const estadoCamas = () => printjson(db.camas.find({ _id: { $in: ["M-20", "M-22"] } }).sort({ _id: 1
}).toArray());
  // Identificadores tomados de la base ya sembrada
  const urg = db.profesionales.findOne({ registro_medico: "RM-0004" });
  const int = db.profesionales.findOne({ registro_medico: "RM-0001" });
  const cef = db.medicamentos.findOne({ codigo: "MED-001" });
  if (!urg || !int || !cef) { print("Faltan datos base: ejecuta primero hce_seed.js"); return; }
  // Limpieza previa por si una ejecución anterior se interrumpió a la mitad
  db.hospitalizaciones.deleteMany({ numero_ingreso: "HOS-2026-0007" });
  db.pacientes.deleteMany({ numero_documento: "1000000099" });
```

```
db.camas.updateMany({ _id: { $in: ["M-20", "M-22"] } }, { $set: { estado: "libre" } });

T("CREATE: insertar paciente, hospitalización y ocupar la cama M-20");
const pacNuevo = ObjectId();
db.pacientes.insertOne({ _id: pacNuevo, tipo_documento: "CC", numero_documento:
"1000000099", nombres: "Sofía", apellidos: "Ramírez Cano", fecha_nacimiento:
F("1992-07-20T00:00"), sexo: "F", telefono: "3000000099", direccion: "Carrera 10 # 5-20, Medellín",
eps: "EPS Ficticia Sur" });
db.hospitalizaciones.insertOne({
  paciente_id: pacNuevo, numero_ingreso: "HOS-2026-0007", estado: "activa",
  admision: { fecha: F("2026-11-05T09:00"), via_ingreso: "urgencias", servicio: "Medicina interna",
cama: "M-20", motivo_consulta: "Diarrea y dolor abdominal de 2 días", profesional_id: urg._id },
  diagnosticos: [{ cie10_codigo: "A09", tipo: "ingreso", fecha: F("2026-11-05T09:30") }],
  tratamientos: [{ medicamento_id: cef._id, dosis: "1 g", via: "IV", frecuencia: "cada 24 h",
fecha_inicio: F("2026-11-05T10:00"), prescribe_id: int._id }]
});
db.camas.updateOne({ _id: "M-20" }, { $set: { estado: "ocupada" } });
estadoCamas();

T("READ: consultar el registro creado");
printjson(db.hospitalizaciones.findOne({ numero_ingreso: "HOS-2026-0007" }, { _id: 0,
numero_ingreso: 1, estado: 1, admision: 1, diagnosticos: 1 }));

T("UPDATE 1: campo simple (teléfono del paciente)");
db.pacientes.updateOne({ _id: pacNuevo }, { $set: { telefono: "3100000099" } });
printjson(db.pacientes.findOne({ _id: pacNuevo }, { _id: 0, nombres: 1, telefono: 1 }));

T("UPDATE 2: campo anidado admision.cama y cambio de estado de las camas");
db.hospitalizaciones.updateOne({ numero_ingreso: "HOS-2026-0007" }, { $set: { "admision.cama":
"M-22" } });
db.camas.updateOne({ _id: "M-20" }, { $set: { estado: "libre" } });
db.camas.updateOne({ _id: "M-22" }, { $set: { estado: "ocupada" } });
printjson(db.hospitalizaciones.findOne({ numero_ingreso: "HOS-2026-0007" }, { _id: 0,
"admision.cama": 1 }));
estadoCamas();

T("UPDATE 3: agregar un elemento al vector diagnosticos ($push)");
db.hospitalizaciones.updateOne({ numero_ingreso: "HOS-2026-0007" }, { $push: { diagnosticos: {
cie10_codigo: "A09", tipo: "principal", fecha: new Date() } } });
printjson(db.hospitalizaciones.findOne({ numero_ingreso: "HOS-2026-0007" }, { _id: 0,
diagnosticos: 1 }));

T("UPDATE 4: campo anidado dentro de un vector (arrayFilters)");
db.hospitalizaciones.updateOne({ numero_ingreso: "HOS-2026-0007" }, { $set: {
"tratamientos.$[t].dosis": "2 g" } }, { arrayFilters: [{ "t.via": "IV" }] });
printjson(db.hospitalizaciones.findOne({ numero_ingreso: "HOS-2026-0007" }, { _id: 0,
tratamientos: 1 }));

T("DELETE lógico (soft delete): marcar como eliminado y liberar la cama");
```

```
  db.hospitalizaciones.updateOne({ numero_ingreso: "HOS-2026-0007" }, { $set: { eliminado: true,
fecha_eliminacion: new Date(), motivo_eliminacion: "Registro creado por error" } });
  db.camas.updateOne({ _id: "M-22" }, { $set: { estado: "libre" } });
  print("Total en la colección: " + db.hospitalizaciones.countDocuments());
  print("Visibles (sin eliminar): " + db.hospitalizaciones.countDocuments({ eliminado: { $ne: true } }));

  T("DELETE definitivo: borrar hospitalización y paciente de prueba");
  db.hospitalizaciones.deleteOne({ numero_ingreso: "HOS-2026-0007" });
  db.pacientes.deleteOne({ _id: pacNuevo });
  print("hospitalizaciones: " + db.hospitalizaciones.countDocuments() + " | pacientes: " +
db.pacientes.countDocuments());
  estadoCamas();
})();
```

Las operaciones se ejecutan con `scripts/hce_crud.js`, después de sembrar la base con `hce_seed.js`. Trabajan sobre un registro de prueba (la hospitalización HOS-2026-0007 y su paciente) para no alterar los datos originales. Al terminar, la base queda con las 6 hospitalizaciones y los 6 pacientes del seed. El script se puede volver a ejecutar: antes de empezar elimina cualquier resto de una ejecución interrumpida.

Operación	Qué hace	Verificar
Create	Inserta un paciente y una hospitalización (con admisión, diagnóstico de ingreso y un tratamiento IV) y marca la cama M-20 como ocupada.	La cama M-20 figura `ocupada`.
Read	Consulta con `findOne` el documento creado.	Se ven admisión y diagnósticos.
Update, campo simple	Cambia el teléfono del paciente con `$set`.	El teléfono pasa a `3100000099`.
Update, campo anidado	Cambia `admision.cama` de M-20 a M-22 y actualiza el estado de ambas camas.	M-20 queda `libre` y M-22 `ocupada`.
Update, vector	Agrega un diagnóstico principal con `$push`.	`diagnosticos` pasa a tener 2 elementos.
Update, dentro de un vector	Cambia la dosis del tratamiento IV de “1 g” a “2 g” con `arrayFilters`.	La dosis queda en `2 g`.

Delete lógico	Marca `eliminado: true` con `fecha_eliminacion` y `motivo_eliminacion`, y libera la cama M-22.	El total es 7 y los visibles (sin eliminar) son 6.
Delete definitivo	Borra con `deleteOne` la hospitalización y luego el paciente de prueba.	Quedan `hospitalizaciones: 6`

Decisión de diseño. El borrado lógico conserva el registro y su trazabilidad, que es lo adecuado para datos clínicos. Por eso todas las consultas filtran `eliminado: { $ne: true }`. El borrado definitivo se reserva para datos de prueba. Al ser MongoDB, las camas se actualizan en un paso aparte de la hospitalización: la consistencia entre ambas la mantiene el script y no el motor.

4. Consultas

Script: https://drive.google.com/file/d/1yqTQQeUz2EOUCsCT5DDeGadQtJoLoFOe/view?usp=drive_link

```
// hce_consultas.js
(function () {
  db = db.getSiblingDB("hce_hospitalizacion");
  // Filtro de soft delete: excluye las hospitalizaciones marcadas como eliminadas
  const NO_ELIMINADO = { $ne: true };
  // Consulta 1: pacientes por diagnóstico
  print("\n=== 1. Pacientes por diagnóstico [4 etapas] | Objetivo: listar quién tiene cada CIE-10 | Valida: $unwind de array embebido y referencia a pacientes ===");
  db.hospitalizaciones.aggregate([
    { $unwind: "diagnosticos" },
    { $match: { "diagnosticos.tipo": "principal", eliminado: NO_ELIMINADO } },
    { $lookup: { from: "pacientes", localField: "paciente_id", foreignField: "_id", as: "pac" } },
    { $group: { _id: "$diagnosticos.cie10_codigo", total: { $sum: 1 }, pacientes: { $addToSet: { $concat: [{ first: "pac.nombres" }, " ", { first: "pac.apellidos" }] } } } }
  ]).forEach(printjson);
  // Consulta 2: evolución clínica cronológica de la hospitalización del primer paciente
  print("\n=== 2. Evolución SOAP cronológica [4 etapas] | Objetivo: línea de tiempo médica del primer paciente | Valida: colección referenciada evoluciones, índice (hospitalizacion_id, fecha) y catálogo profesionales ===");
  const p1 = db.pacientes.find().sort({ numero_documento: 1 }).limit(1).toArray()[0];
  const h1 = db.hospitalizaciones.findOne({ paciente_id: p1._id, eliminado: NO_ELIMINADO });
```

```
print("Hospitalización: " + h1.numero_ingreso);
db.evoluciones.aggregate([
  { $match: { hospitalizacion_id: h1._id } },
  { $sort: { fecha: 1 } },
  { $lookup: { from: "profesionales", localField: "profesional_id", foreignField: "_id", as: "prof" } },
  { $project: { _id: 0, fecha: 1, profesional: { $concat: [{ first: "prof.nombres" }, " ", { first: "
prof.apellidos" }] }, subjetivo: 1, objetivo: 1, analisis: 1, plan: 1 } }
]).forEach(printjson);
// Consulta 3: medicamentos asignados a un paciente
print("\n=== 3. Medicamentos de un paciente [4 etapas] | Objetivo: tratamientos con datos del
catálogo | Valida: tratamientos embebidos con medicamento_id referenciado ===");
print("Paciente: " + p1.nombres + " " + p1.apellidos);
db.hospitalizaciones.aggregate([
  { $match: { paciente_id: p1._id, eliminado: NO_ELIMINADO } },
  { unwind: "tratamientos" },
  { $lookup: { from: "medicamentos", localField: "tratamientos.medicamento_id", foreignField: "_id",
as: "med" } },
  { $project: { _id: 0, ingreso: "$numero_ingreso", medicamento: { first: "med.nombre_generico" },
dosis: "tratamientos.dosis", via: "tratamientos.via", frecuencia: "
tratamientos.frecuencia", fecha_inicio: "tratamientos.fecha_inicio", fecha_fin:
"$tratamientos.fecha_fin" } }
]).forEach(printjson);
// Consulta 4: hospitalizaciones activas con último signo vital
print("\n=== 4. Activas con último signo vital [4 etapas] | Objetivo: estado actual de pacientes
hospitalizados | Valida: $lookup con sub-pipeline sobre colección referenciada e índice
(hospitalizacion_id, fecha desc) ===");
db.hospitalizaciones.aggregate([
  { $match: { estado: "activa", eliminado: NO_ELIMINADO } },
  { $lookup: { from: "signos_vitales", let: { hid: "$_id" }, pipeline: [
    { $match: { $expr: { eq: ["hospitalizacion_id", "$$hid"] } } },
    { $sort: { fecha: -1 } },
    { $limit: 1 },
    { $project: { _id: 0, fecha: 1, fc: 1, fr: 1, temp: 1, sat_o2: 1, ta_sis: 1, ta_dia: 1 } }
  ], as: "ultimo_signo_vital" } },
  { $unwind: { path: "$ultimo_signo_vital", preserveNullAndEmptyArrays: true } },
  { $project: { _id: 0, numero_ingreso: 1, estado: 1, servicio: "$admision.servicio", cama:
"$admision.cama", ultimo_signo_vital: 1 } }
]).forEach(printjson);
// Consulta 5: explain de búsqueda por código CIE-10
print("\n=== 5. Explain por diagnosticos.cie10_codigo | Objetivo: comprobar uso de índice | Valida:
índice multikey sobre array embebido de diagnósticos ===");
```

```
  const ex = db.hospitalizaciones.find({ "diagnosticos.cie10_codigo": "J18.9"
}).explain("executionStats");
  const plan = ex.queryPlanner.winningPlan;
  const nodo = plan.inputStage || (plan.queryPlan && plan.queryPlan.inputStage) || plan;
  const raiz = plan.queryPlan || plan;
  printjson({
    "winningPlan.stage": raiz.stage,
    indexName: nodo.indexName || (nodo.inputStage && nodo.inputStage.indexName),
    totalDocsExamined: ex.executionStats.totalDocsExamined,
    nReturned: ex.executionStats.nReturned
  });
  // Consulta 6: vista de auditoría sin identidad
  print("\n=== 6. Auditoría sin identidad [3 etapas] | Objetivo: datos clínicos anonimizados | Valida:
separación de privacidad entre hospitalizaciones y pacientes ===");
  db.hospitalizaciones.aggregate([
    { $match: { eliminado: NO_ELIMINADO } },
    { $project: { _id: 0, numero_ingreso: 1, estado: 1, servicio: "$admision.servicio", fecha_admision:
"admision.fecha", fecha_alta: "alta.fecha", diagnosticos: 1 } },
    { $sort: { numero_ingreso: 1 } }
  ]).forEach(printjson);
  // Consulta 7 (propia: Santiago Padierna): ocupación de camas por servicio
  print("\n=== 7. Ocupación de camas por servicio [3 etapas] | Objetivo: porcentaje de ocupación
por servicio | Valida: catálogo de camas, $group con $cond y cálculo con subtract/divide/$multiply
===");
  db.camas.aggregate([
    { $group: { _id: "$servicio", total_camas: { $sum: 1 }, ocupadas: { $sum: { $cond: [{ eq: ["estado",
"ocupada"] }, 1, 0] } } } },
    { $project: { _id: 0, servicio: "$_id", total_camas: 1, ocupadas: 1, libres: { subtract: ["
total_camas", "$ocupadas"] }, ocupacion_pct: { $round: [{ $multiply: [{ divide: ["ocupadas",
"$total_camas"] }, 100] }, 1] } } },
    { $sort: { ocupacion_pct: -1, servicio: 1 } }
  ]).forEach(printjson);
})();
```

Consulta 1. Pacientes por diagnóstico

Objetivo: listar, por código CIE-10 principal, quiénes lo tienen.

Valida: $unwind sobre array embebido y $lookup a catálogo y pacientes.

Verificar: J18.9 debe agrupar a Juan Camilo Pérez Gómez y María Fernanda Gómez Ruiz.

Resultado:

{ _id: 'J18.9', total: 2, pacientes: ['María Fernanda Gómez Ruiz', 'Juan Camilo Pérez Gómez'] }

```
=== 1. Pacientes por diagnóstico [4 etapas] | Objetivo: listar quién tiene cada CIE-10 | Valida: $unwind de array embebido y referencia a pacientes ===
{ _id: 'J44.1', total: 1, pacientes: [ 'Pedro Pablo Salazar Henao' ] }
{ _id: 'N39.0', total: 1, pacientes: [ 'Ana Sofía Cardona Londoño' ] }
{
  _id: 'J18.9',
  total: 2,
  pacientes: [ 'Juan Camilo Pérez Gómez', 'María Fernanda Gómez Ruiz' ]
}
{ _id: 'I50.9', total: 1, pacientes: [ 'Luis Alberto Torres Mora' ] }
{ _id: 'E11.1', total: 1, pacientes: [ 'Valentina Ríos Arango' ] }
```

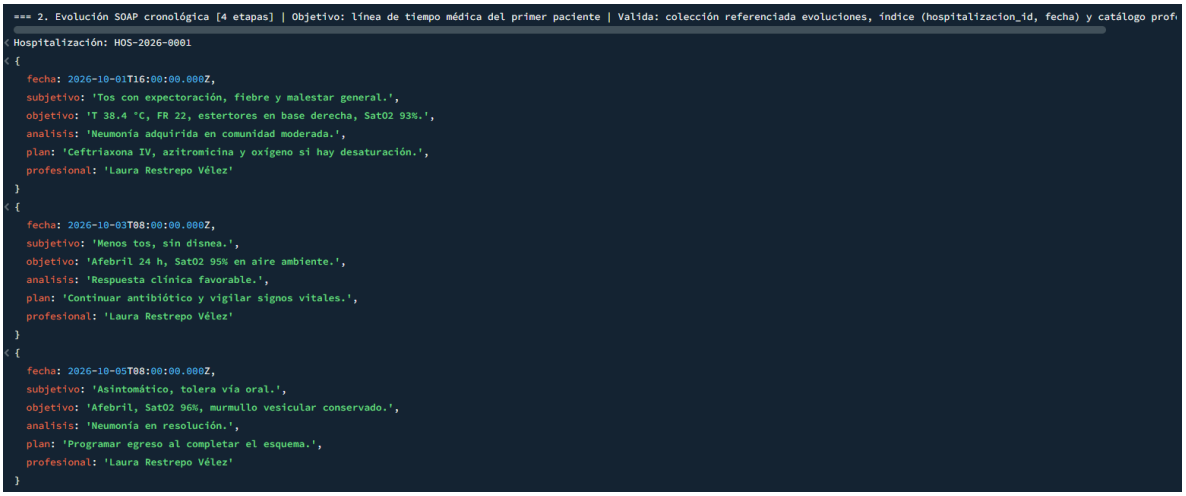
Consulta 2. Evolución SOAP cronológica de HOS-2026-0001.

Objetivo: línea de tiempo médica del primer paciente, con nombre del profesional.

Valida: colección referenciada evoluciones, índice (hospitalizacion_id, fecha) y catálogo profesionales.

Resultado:

{
  fecha: 2026-10-01T16:00:00.000Z,
  subjetivo: 'Tos con expectoración, fiebre y malestar general.',
  objetivo: 'T 38.4 °C, FR 22, estertores en base derecha, SatO2 93%.',
  analisis: 'Neumonía adquirida en comunidad moderada.',
  plan: 'Ceftriaxona IV, azitromicina y oxígeno si hay desaturación.',
  profesional: 'Laura Restrepo Vélez'
}



Consulta 3. Medicamentos de un paciente.

Objetivo: tratamientos con nombre genérico, dosis, vía, frecuencia y fechas.

Valida: tratamientos embebidos con medicamento_id referenciado.

Resultado:

{

ingreso: 'HOS-2026-0001',

medicamento: 'Ceftriaxona',

dosis: '1 g',

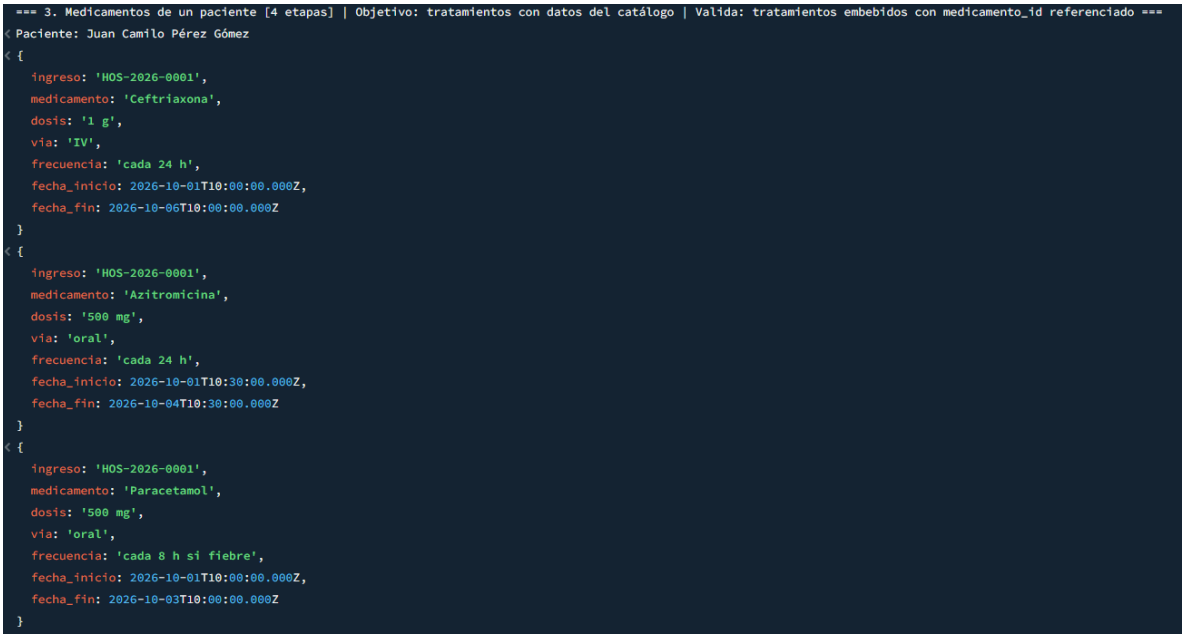
via: 'IV',

frecuencia: 'cada 24 h',

fecha_inicio: 2026-10-01T10:00:00.000Z,

fecha_fin: 2026-10-06T10:00:00.000Z

}



Consulta 4. Hospitalizaciones activas con último signo vital.

Objetivo: estado actual de los pacientes hospitalizados.

Valida: $lookup con sub-pipeline (sort descendente y limit 1) sobre colección referenciada.

Resultado:

{
numero_ingreso: 'HOS-2026-0003',
estado: 'activa',
ultimo_signo_vital: {
fecha: 2026-10-25T08:00:00.000Z,
fc: 80,
fr: 18,
temp: 36.6,
sat_o2: 95,
ta_sis: 118,
ta_dia: 72

```
  },
  servicio: 'Cardiología',
  cama: 'C-05'
}
```

```
=== 4. Activas con último signo vital | Objetivo: estado actual de pacientes hospitalizados | Valida: $lookup con sub-pipeline sobre colección referenciada e índice (hospitalizacion_id, fecha desc) ===
{
  numero_ingreso: 'HOS-2026-0003',
  estado: 'activa',
  ultimo_signo_vital: {
    fecha: 2026-10-25T08:00:00.000Z,
    fc: 80,
    fr: 18,
    temp: 36.6,
    sat_o2: 95,
    ta_sis: 118,
    ta_dia: 72
  },
  servicio: 'Cardiología',
  cama: 'C-05'
}
```

Consulta 5. `explain("executionStats")` por `diagnosticos.cie10_codigo`.

Objetivo: comprobar el uso de índice.

Valida: índice multikey sobre el array embebido de diagnósticos.

Resultado (winningPlan.stage, indexName, totalDocsExamined, nReturned):

```
{
  'winningPlan.stage': 'FETCH',
  indexName: 'diagnosticos.cie10_codigo_1',
  totalDocsExamined: 2,
  nReturned: 2
}
```

```
=== 5. Explain por diagnosticos.cie10_codigo | Objetivo: comprobar uso de índice | Valida: índice multikey sobre array embebido de diagnósticos ===
{
  'winningPlan.stage': 'FETCH',
  indexName: 'diagnosticos.cie10_codigo_1',
  totalDocsExamined: 2,
  nReturned: 2
}
```

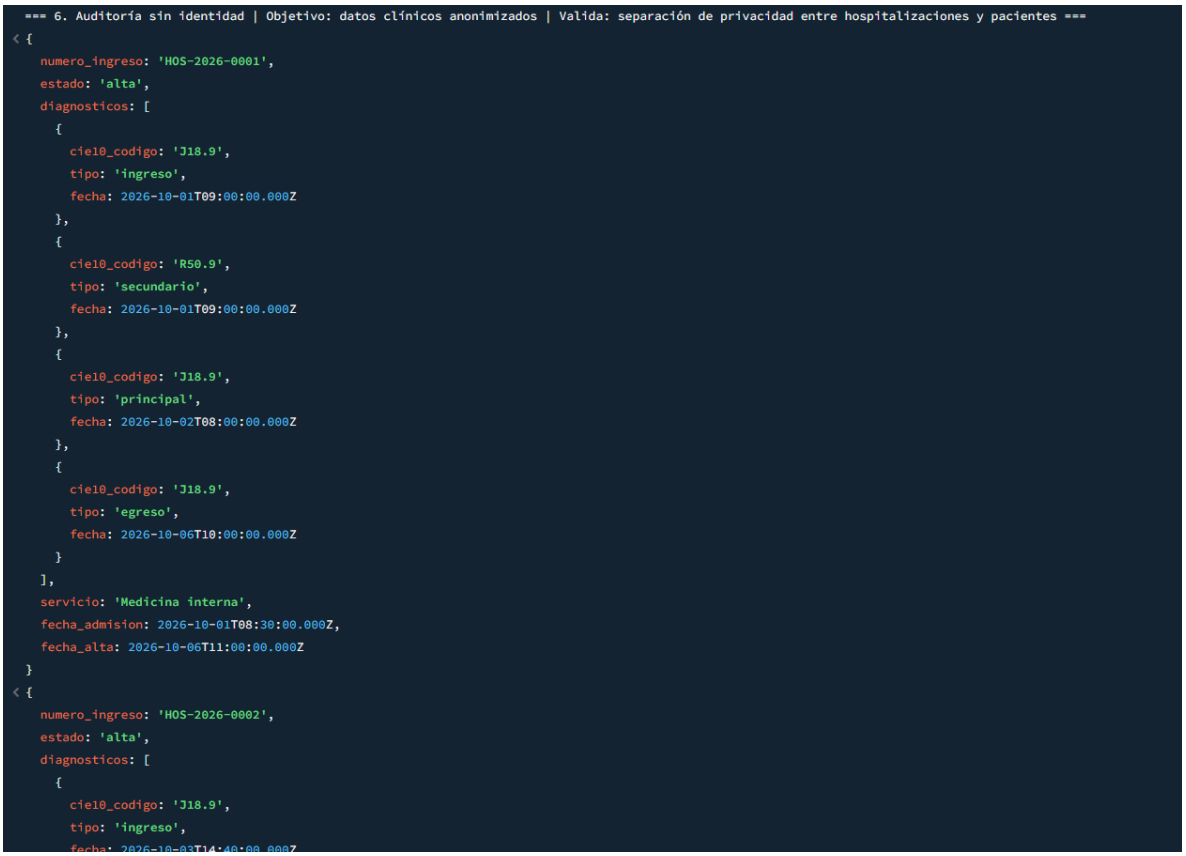
Consulta 6. Vista de auditoría sin identidad.

Objetivo: exponer datos clínicos sin paciente_id ni datos de pacientes.

Valida: separación de privacidad entre colecciones.

Resultado:

```
{
  numero_ingreso: 'HOS-2026-0001',
  estado: 'alta',
  diagnosticos: [
    { cie10_codigo: 'J18.9', tipo: 'ingreso', fecha: 2026-10-01T09:00:00.000Z },
    { cie10_codigo: 'R50.9', tipo: 'secundario', fecha: 2026-10-01T09:00:00.000Z },
    { cie10_codigo: 'J18.9', tipo: 'principal', fecha: 2026-10-02T08:00:00.000Z },
    { cie10_codigo: 'J18.9', tipo: 'egreso', fecha: 2026-10-06T10:00:00.000Z }
  ],
  servicio: 'Medicina interna',
  fecha_admision: 2026-10-01T08:30:00.000Z,
  fecha_alta: 2026-10-06T11:00:00.000Z
}
```



Consulta 7. Ocupación de camas por servicio.

Objetivo: calcular el porcentaje de ocupación de camas por servicio.

Válida: Cardiología debe tener 1 de 2 camas ocupadas (50 %), por HOS-2026-0003 en C-05.

Resultado:

{ servicio: 'Cardiología', total_camas: 2, ocupadas: 1, libres: 1, ocupacion_pct: 50 }

{ servicio: 'Medicina interna', total_camas: 7, ocupadas: 0, libres: 7, ocupacion_pct: 0 }

{ servicio: 'Neumología', total_camas: 2, ocupadas: 0, libres: 2, ocupacion_pct: 0 }



5. Evidencia de IA

Herramienta	Prompt (resumen)	Aporte	Corrección manual
Claude	Diseñar el modelo híbrido: tablas, JSON de ejemplo, `$jsonSchema`, índices, diagrama y comparación embebido vs referenciado	Esquema, validadores, índices y diagrama	MongoDB no valida referencias, por lo que se agregó verificación de integridad. La vista “sin identidad” solo seudonimiza, no anonimiza

Claude	Script único de creación y carga con datos ficticios (`hce_seed.js`)	Colecciones, índices, datos y verificación	Se pidió drop previo para re-ejecutarlo, `getSiblingDB` y función autoejecutable. Se añadió el chequeo de huérfanos, el de estado vs alta y la colección `camas` que pide el enunciado
Claude	Operaciones CRUD con soft delete (`hce_crud.js`)	Inserción, actualización simple, anidada y en vector, y borrado lógico y definitivo	El primer borrador usaba el estado “en observación”, que el validador rechaza, y un código CIE-10 fuera del catálogo. Se corrigieron y se separó el CRUD del seed
Claude	7 consultas con `$lookup`, sub-pipeline y `explain` (`hce_consultas.js`)	Consultas y comprobación de que el índice se usa (IXSCAN)	Se redujeron a 2–4 etapas, se excluyen los registros con soft delete y se agregó la consulta propia de ocupación de camas

Historial de conversación con la IA:

- Diseño del modelo y carga: https://claude.ai/share/4c8a26cd-19fe-4ddc-aa0e-0e5a25e87a35

6. Reflexión ética

Los datos de salud son sensibles según la Ley 1581 de 2012, que exige autorización expresa y tratamiento reforzado. La Resolución 1995 de 1999 da a la historia clínica carácter reservado, exige su integridad y su conservación por 20 años. Las decisiones de modelado afectan ambos marcos.

- **a) Privacidad.** Separar pacientes de hospitalizaciones solo seudonimiza: `paciente_id` permite reidentificar a quien acceda a ambas colecciones. Además, `motivo_consulta`, las notas y las evoluciones son texto libre y pueden contener datos identificables. Por eso la consulta 6 es seudonimizada, no anónima.
- **b) Seguridad.** La separación permite roles de mínimo privilegio por colección, por ejemplo auditoría sin acceso a pacientes. Requiere cifrado en tránsito y en reposo y registro de accesos.
- **c) Consistencia.** MongoDB no impone integridad referencial. Además, una operación que toca varias colecciones, como cambiar la cama de un paciente (hospitalización y camas), no es atómica sin transacción multi-documento. En el CRUD la coordina el script. Las mitigaciones son la verificación de integridad, las transacciones y la validación en la aplicación.
- **d) Embebido vs referenciado.** Embeber da atomicidad por documento, pero los arrays sin tope amenazan el límite de 16 MB. Por eso lo que crece (signos vitales, notas y evoluciones) se referencia, aceptando perder esa atomicidad.
- **e) Trazabilidad y borrado.** El soft delete conserva el registro con fecha y motivo de eliminación, coherente con la integridad que exige la Resolución 1995. El borrado definitivo se reserva a datos de prueba. Aún falta registrar quién modifica qué y cuándo: campos de auditoría, versiones y registro de lecturas.

6. División del trabajo

Integrante: Santiago Padierna Suarez (trabajo individual).

Rol	Tareas
Diseño del modelo	Esquema, justificación embebido vs referenciado, diagrama

Implementación y carga	Colecciones, validadores, índices, datos ficticios y verificación de integridad
CRUD	Inserción, actualización, soft delete y borrado definitivo
Consultas	Las 7 consultas y el análisis de `explain`
Documentación, ética e IA	README, repositorio en GitHub, capturas, reflexión ética y evidencia de IA